

M4_Regressió_Logística

January 12, 2026

1 Regressió Logística

1.0.1 Diferències entre Regressió i Classificació

La **regressió lineal** (algoritme vist fins ara) s'utilitza per predir **valors numèrics continus**, com el preu d'un diamant amb unes determinades característiques.

El seu model segueix una relació lineal entre les variables ($y = wx + b$) on l'output pot prendre valors des de $-\infty$ a $+\infty$.

La **regressió logística** ens permet classificar en categories, com per exemple, determinar si un correu és "spam" o "no spam".

El resultat s'expressa com una probabilitat compresa entre 0 i 1, calculada amb la funció sigmoide.

En l'exemple que veurem a continuació, volem poder predir la possibilitat que un tumor de mama/pit sigui maligne o benigne.

Quan la classificació es fa entre dos categories, com és en aquest cas, parlarem de problema de **classificació binària**.

Tot i que té el nom de regressió, la regressió logística és un algoritme de **classificació**.

En lloc de predir valors continus com la regressió lineal, la regressió logística calcula la probabilitat que una observació pertanyi a una classe determinada utilitzant la funció sigmoide

1.0.2 Funció Sigmoide

Compleix que: * Output sempre entre 0 i 1 * Si $x \rightarrow +\infty$, $\sigma(x) \rightarrow 1$ * Si $x \rightarrow -\infty$, $\sigma(x) \rightarrow 0$ * Si $x = 0$, $\sigma(x) = 0.5$

1.0.3 Exemple de classificació amb dades de càncer de mama

Carreguem les dades de càncer de mama:

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, classification_report,
                             accuracy_score, precision_score, recall_score,
                             f1_score, ConfusionMatrixDisplay)
```

```
[ ]: breast_cancer = load_breast_cancer()
df = pd.DataFrame(breast_cancer.data, columns=breast_cancer.feature_names)
df['target'] = breast_cancer.target
```

Fem una primera inspecció de les dades:

```
[ ]: df.head()
```

```
[ ]:
mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0          17.99         10.38         122.80      1001.0         0.11840
1          20.57         17.77         132.90      1326.0         0.08474
2          19.69         21.25         130.00      1203.0         0.10960
3          11.42         20.38          77.58       386.1         0.14250
4          20.29         14.34         135.10     1297.0         0.10030

mean compactness  mean concavity  mean concave points  mean symmetry  \
0          0.27760          0.3001          0.14710          0.2419
1          0.07864          0.0869          0.07017          0.1812
2          0.15990          0.1974          0.12790          0.2069
3          0.28390          0.2414          0.10520          0.2597
4          0.13280          0.1980          0.10430          0.1809

mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0          0.07871  ...          17.33          184.60        2019.0
1          0.05667  ...          23.41          158.80        1956.0
2          0.05999  ...          25.53          152.50        1709.0
3          0.09744  ...          26.50           98.87         567.7
4          0.05883  ...          16.67          152.20        1575.0

worst smoothness  worst compactness  worst concavity  worst concave points  \
0          0.1622          0.6656          0.7119          0.2654
1          0.1238          0.1866          0.2416          0.1860
2          0.1444          0.4245          0.4504          0.2430
3          0.2098          0.8663          0.6869          0.2575
4          0.1374          0.2050          0.4000          0.1625

worst symmetry  worst fractal dimension  target
0          0.4601          0.11890          0
1          0.2750          0.08902          0
2          0.3613          0.08758          0
3          0.6638          0.17300          0
4          0.2364          0.07678          0
```

[5 rows x 31 columns]

```
[ ]: print(f"Nombre de mostres: {len(df)}")
      print(f"Nombre de 'features': {len(df.columns)-1}")
```

Nombre de mostres: 569

Nombre de 'features': 30

El valors de target ja estan regularitzats a valors numèrics (0=Maligne, 1=Benigne):

```
[ ]: df['target'].value_counts()
```

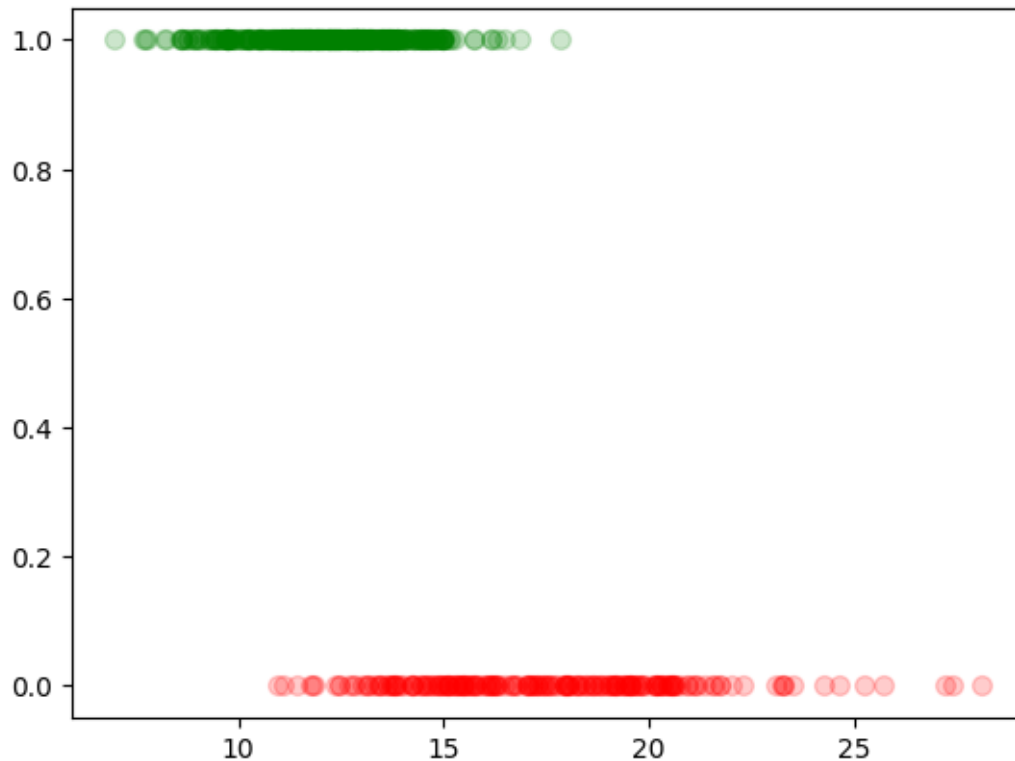
```
[ ]: target
      1    357
      0    212
      Name: count, dtype: int64
```

Intentem fer una regressió logística amb la feature 'mean radius' (tenint en compte un sol "feature"):

```
[ ]: X = df[['mean radius']].values
      y = df['target'].values

      plt.scatter(X[y==0], y[y==0], color='red', alpha=0.2, label='Maligne (0)', s=50)
      plt.scatter(X[y==1], y[y==1], color='green', alpha=0.2, label='Benigne (1)', s=50)
```

```
[ ]: <matplotlib.collections.PathCollection at 0x7b016a4bcef0>
```



```
[ ]: model = LogisticRegression(solver='liblinear')
model.fit(X, y)

# Crear corba sigmoide per visualitzar
x_plot = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
sigmoid_pred = model.predict_proba(x_plot)[:, 1]

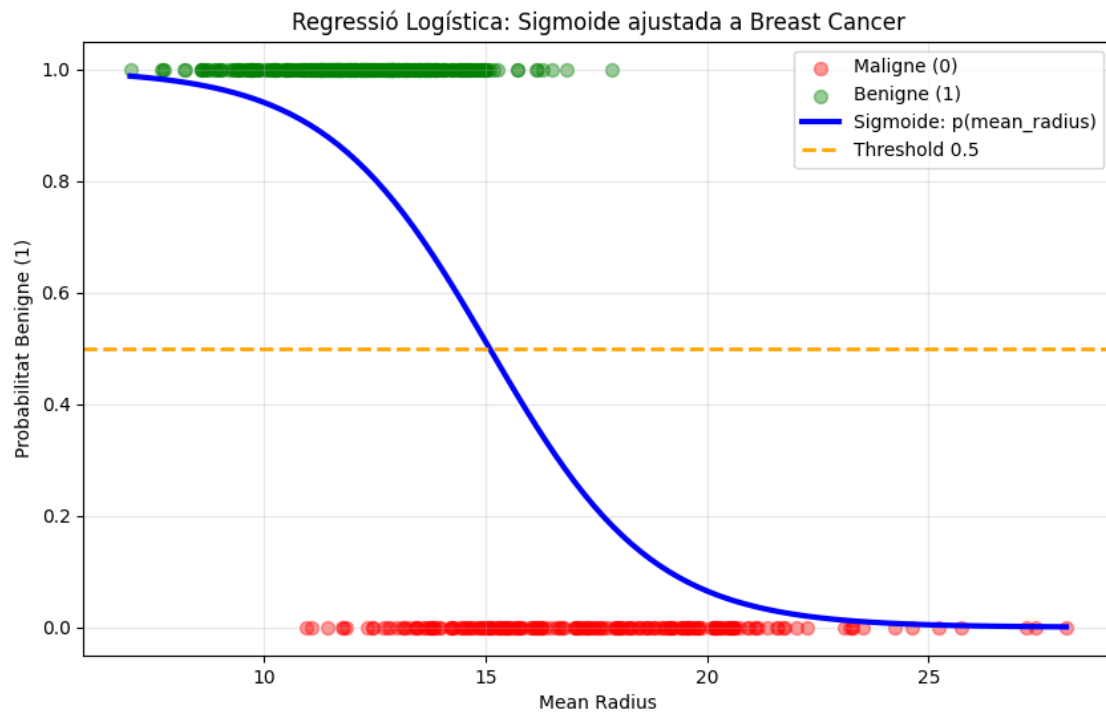
plt.figure(figsize=(10, 6))
plt.scatter(X[y==0], y[y==0], color='red', alpha=0.4, label='Maligne (0)', s=50)
plt.scatter(X[y==1], y[y==1], color='green', alpha=0.4, label='Benigne (1)', s=50)

# CORBA SIGMOIDE (funció logística completa)
plt.plot(x_plot, sigmoid_pred, color='blue', linewidth=3,
         label='Sigmoide: p(mean_radius)')

plt.axhline(y=0.5, color='orange', linestyle='--', linewidth=2,
            label='Threshold 0.5')

plt.xlabel('Mean Radius')
plt.ylabel('Probabilitat Benigne (1)')
plt.title('Regressió Logística: Sigmoide ajustada a Breast Cancer')
```

```
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



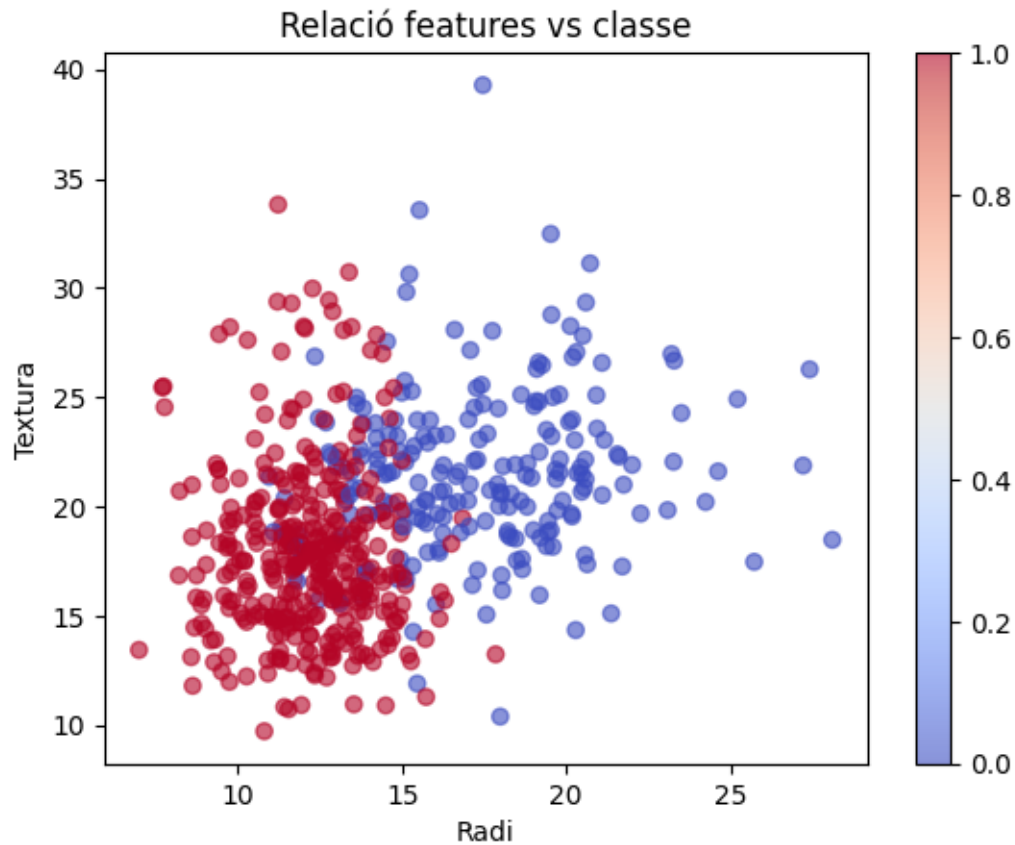
Obtenim la corba sigmoide perfecta superposada als teus punts. La línia taronja (0.5) marca exactament on el model canvia de classe:

$x = 15: p(x) = 0.50$ (frontera)

1.1 Regressió logística múltiple

Quan en el model introduïm més d'una variable d'entrada parlem d'una regressió logística múltiple. El resultat es pot entendre com una divisió entre regions.

```
[ ]: plt.scatter(df['mean radius'], df['mean texture'], c = df['target'],
               cmap='coolwarm', alpha=0.6)
plt.xlabel('Radi')
plt.ylabel('Textura')
plt.title('Relació features vs classe')
plt.colorbar()
plt.show()
```



```
[ ]: X = df[['mean radius', 'mean texture']].values
y = df['target'].values

model = LogisticRegression(solver='liblinear')
model.fit(X, y)

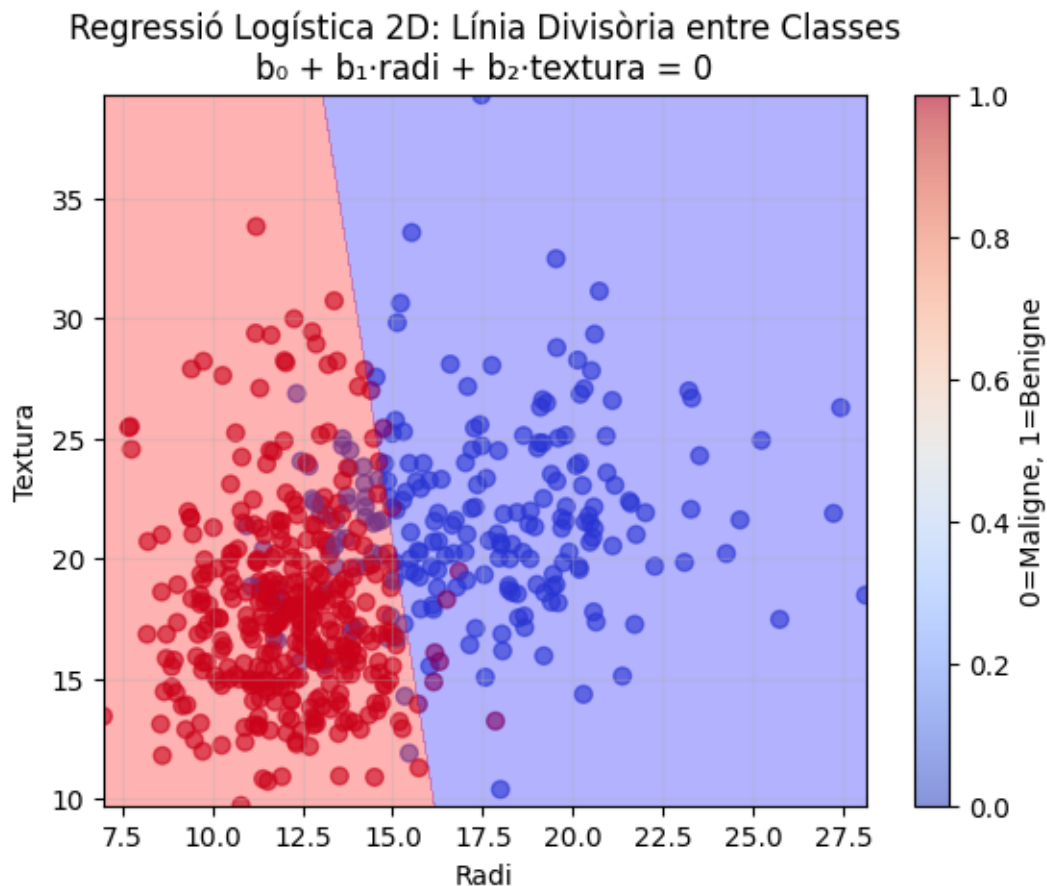
xx, yy = np.meshgrid(np.linspace(X[:,0].min(), X[:,0].max(), 100),
                     np.linspace(X[:,1].min(), X[:,1].max(), 100))
Z = model.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
Z = Z.reshape(xx.shape)

scatter = plt.scatter(df['mean radius'], df['mean texture'],
                     c=df['target'], cmap='coolwarm', alpha=0.6, s=40)
plt.colorbar(scatter, label='0=Maligne, 1=Benigne')

plt.contourf(xx, yy, Z, levels=[0, 0.5, 1], alpha=0.3,
             colors=['blue', 'red'], linestyles='dashed')
```

```
#plt.contour(xx, yy, Z, levels=[0.5], colors='black', linewidths=1,
↳linestyles='-.')

plt.xlabel('Radi')
plt.ylabel('Textura')
plt.title('Regressió Logística 2D: Línia Divisòria entre Classes\n'
          'b + b · radi + b · textura = 0')
plt.grid(True, alpha=0.3)
plt.show()
```



De fet per a una variable també ho podem representar com una “línia” divisòria:

```
[ ]: X = df[['mean radius']].values
y = df['target'].values

model = LogisticRegression(solver='liblinear')
model.fit(X, y)

# Crear corba sigmoide per visualitzar
```

```

x_plot = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
sigmoid_pred = model.predict_proba(x_plot)[:, 1]

plt.figure(figsize=(10, 6))
plt.scatter(X[y==0], y[y==0], color='red', alpha=0.4, label='Maligne (0)', s=50)
plt.scatter(X[y==1], y[y==1], color='green', alpha=0.4, label='Benigne (1)', s=50)

# CORBA SIGMOIDE (funció logística completa)
plt.plot(x_plot, sigmoid_pred, color='blue', linewidth=3,
         label='Sigmoide: p(radi)')

plt.axhline(y=0.5, color='orange', linestyle='--', linewidth=2,
            label='Threshold 0.5')

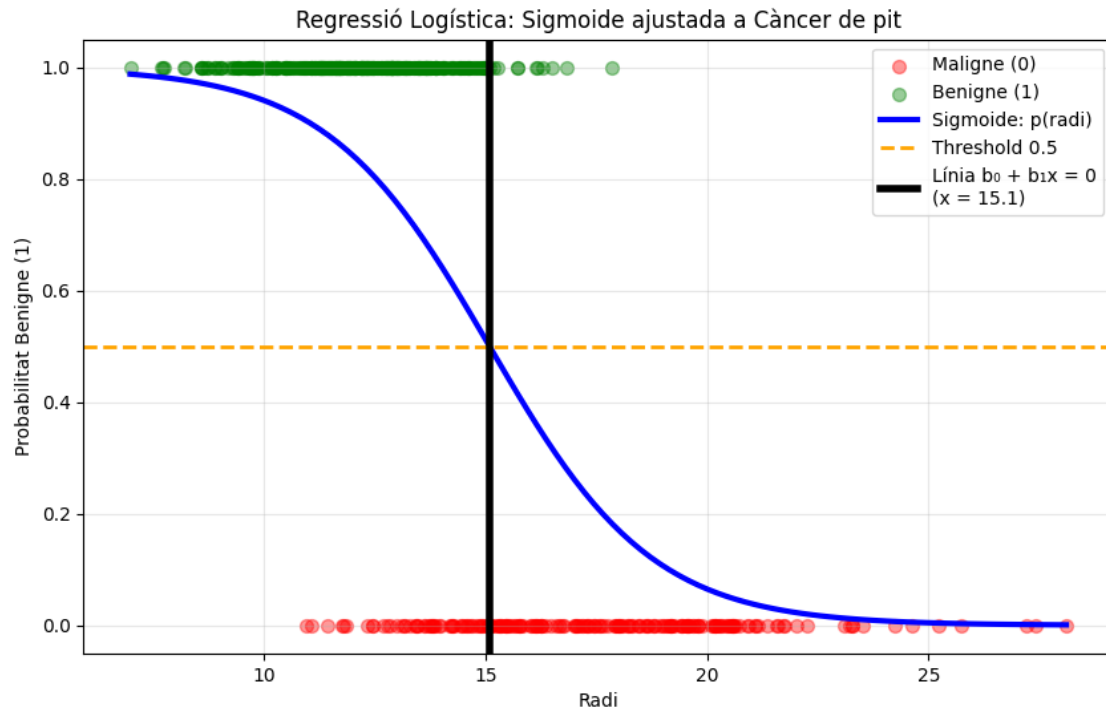
# LÍNIA  $b + b \cdot x = 0$ 

# COEFICIENTS (RESULTATS REALS)
b0 = model.intercept_[0]      # b
b1 = model.coef_[0][0]       # b

# PUNT on  $b + b \cdot x = 0$  (la "línia" en 1D)
punt_decisio = -b0 / b1
plt.axvline(x=punt_decisio, color='black', linewidth=4,
            label=f'Línia  $b + b \cdot x = 0$  (x = {punt_decisio:.1f})')

plt.xlabel('Radi')
plt.ylabel('Probabilitat Benigne (1)')
plt.title('Regressió Logística: Sigmoide ajustada a Càncer de pit')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

1 variable: Regressió logística simple ($b_0 + b_1 \cdot x$)

2+ variables: Regressió logística múltiple ($b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_n \cdot x_n$)

1.2 Entrenament del model i lectura de resultats

A continuació entrenarem el model amb totes les variables disponibles.

```
[ ]: y = df['target']
X = df.drop('target', axis=1)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
[ ]: model = LogisticRegression(solver='liblinear', max_iter=10000, random_state=42)
    # solver utilitzat quan tenim 2 classes (classificador binari)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Veiem el resultat per a les 4 primeres mostres:

```
[ ]: y_proba = model.predict_proba(X_test)
print(y_proba[:4])
```

```
[[1.00000000e+00 1.95855858e-12]
 [3.52718528e-04 9.99647281e-01]]
```

```
[8.92285401e-01 1.07714599e-01]
[3.48199872e-01 6.51800128e-01]]
```

Que té la següent interpretació:

```
[ ]: classes = ['Maligne', 'Benigne']
for i, prob in enumerate(y_proba[:4]):
    print(f"Pacient {i+1}: {classes[0]} {prob[0]:.2%} | {classes[1]} {prob[1]:.2%}")
```

Pacient 1: Maligne 100.00% | Benigne 0.00%

Pacient 2: Maligne 0.04% | Benigne 99.96%

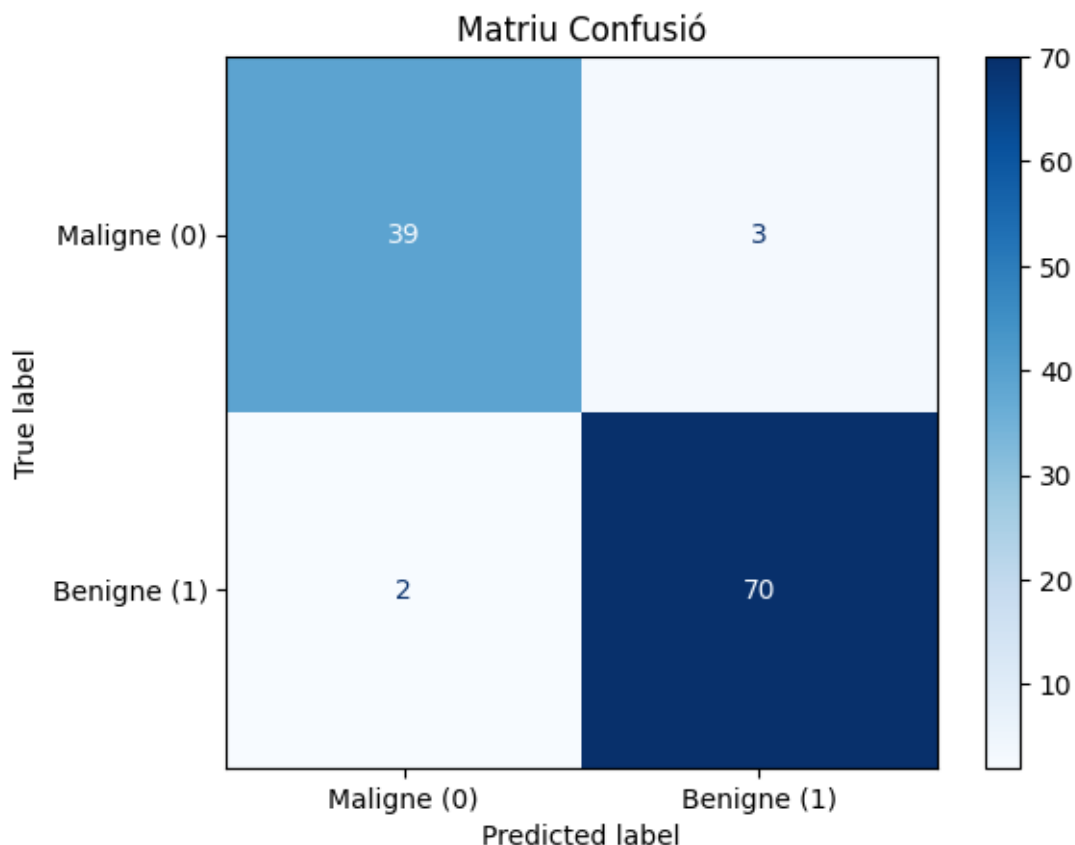
Pacient 3: Maligne 89.23% | Benigne 10.77%

Pacient 4: Maligne 34.82% | Benigne 65.18%

##Matriu de Confusió

Però ara ens falta saber com de bons són aquests resultats i mesurar/calcular els errors:

```
[ ]: cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Maligne',
    ↪(0)', 'Benigne (1)']) # Ordre real: [positiu, negatiu]
disp.plot(cmap='Blues')
plt.title('Matriu Confusió')
plt.show()
```



Ho podem llegir de la següent manera:

```
[ ]: print("Matriu original (sklearn ordre [0,1]):\n", cm)

tn, fp, fn, tp = cm.ravel()

print("\nCONVENCIÓ PÚBLICA (Benigne=Positiu)")
print(f"Negatiu Verdaders (TN): {tn}      ← Maligne real/predit Maligne")
print(f"Falsos Positiu (FP): {fp}      ← Maligne real/predit Benigne")
print(f"Falsos Negatiu (FN): {fn}      ← Benigne real/predit Maligne")
print(f"Positiu Verdaders (TP): {tp}      ← Benigne real/predit Benigne")

print("\nCONVENCIÓ MÈDICA (Maligne=Positiu)")
# Per Maligne(0)=Positiu: transposa lògicament
tp_maligne = cm[0,0]      # Maligne real/predit Maligne
fn_maligne = cm[0,1]      # Maligne real/predit Benigne (error greu!)
fp_maligne = cm[1,0]      # Benigne real/predit Maligne (alarma falsa)
tn_maligne = cm[1,1]      # Benigne real/predit Benigne

print(f"Positiu Verdaders (TP): {tp_maligne}      ← Càncer detectat!")
```

```
print(f"Falsos Negatius (FN): {fn_maligne} ← Càncer no detectat!")
print(f"Falsos Positius (FP): {fp_maligne} ← Alarma falsa")
print(f"Negatius Verdaders (TN): {tn_maligne}")
```

Matriu original (sklearn ordre [0,1]):

```
[[39  3]
 [ 2 70]]
```

CONVENCIÓ PÚBLICA (Benigne=Positiu)

```
Negatius Verdaders (TN): 39 ← Maligne real/predit Maligne
Falsos Positius (FP): 3 ← Maligne real/predit Benigne
Falsos Negatius (FN): 2 ← Benigne real/predit Maligne
Positius Verdaders (TP): 70 ← Benigne real/predit Benigne
```

CONVENCIÓ MÈDICA (Maligne=Positiu)

```
Positius Verdaders (TP): 39 ← Càncer detectat!
Falsos Negatius (FN): 3 ← Càncer no detectat!
Falsos Positius (FP): 2 ← Alarma falsa
Negatius Verdaders (TN): 70
```

1.2.1 Mètriques d'avaluació

Les següents mètriques avaluen el rendiment d'un model de classificació binària basant-se en la matriu de confusió (TN, FP, FN, TP).

Es mostren el % per facilitar-ne la interpretació.

Accuracy: Proporció total de prediccions correctes.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: De les prediccions positives ("Maligne"), quantes ho són realment.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensibilitat): dels casos positius reals ("Maligne"), quina proporció es detecta.

$$\text{Precision} = \frac{TP}{TP + FN}$$

F1- Score: mitjana harmònica entre Precision i Recall. Ideal per detectar quan les classes estan descompensades.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Per tant un bon model estarà marcat per:

- Accuracy > 0.90: Rendiment general bo.

- Recall alt: Detecció de tots els positius reals (crític en medicina).
- Precision alta: Confiança en els positius predits. Tots els positius predits són positius reals.
- F1-Score > 0.90: indica que les classes estan balancejades.

```
[ ]: accuracy = accuracy_score(y_test, y_pred)
precision_m = precision_score(y_test, y_pred, average='binary', pos_label=0) #
    ↳indiquem l'equiqueta de 'positiu'
recall_m    = recall_score(y_test, y_pred, average='binary', pos_label=0) #
    ↳indiquem l'equiqueta de 'positiu'
f1_m        = f1_score(y_test, y_pred, average='binary', pos_label=0) #
    ↳indiquem l'equiqueta de 'positiu'

print(f"Accuracy:  {accuracy:.3f} ({accuracy*100:.1f}%)")
print(f"Precision: {precision_m:.3f} ({precision_m*100:.1f}%)")
print(f"Recall:    {recall_m:.3f} ({recall_m*100:.1f}%)")
print(f"F1-Score:  {f1_m:.3f} ({f1_m*100:.1f}%)")
```

```
Accuracy:  0.956 (95.6%)
Precision: 0.951 (95.1%)
Recall:    0.929 (92.9%)
F1-Score:  0.940 (94.0%)
```